Attention Is All You Need

Ashish Vaswani* Google Brain avaswani@google.com

Noam Shazeer* Google Brain noam@google.com

Niki Parmar* Google Research nikip@google.com

Jakob Uszkoreit* Google Research usz@google.com

Llion Jones* Google Research llion@google.com

Aidan N. Gomez* † University of Toronto aidan@cs.toronto.edu

Łukasz Kaiser* Google Brain lukaszkaiser@google.com

Illia Polosukhin* ‡ illia.polosukhin@gmail.com

Member: 성지석, 최재규, 허다운

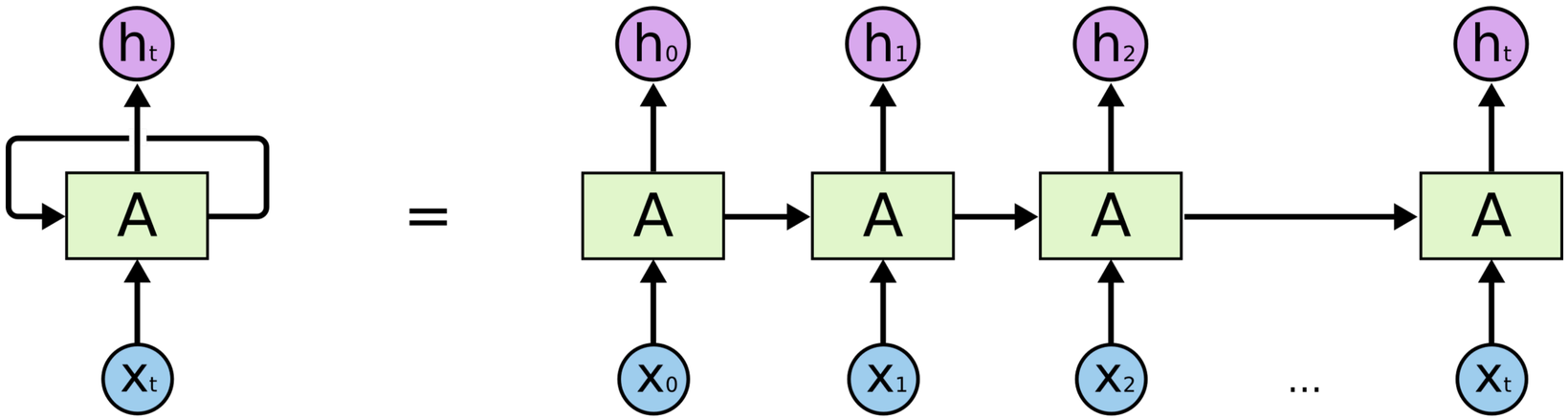
Presenter: 허다운

Contents

1. Introduction & Background

1. Recurrent Neural Network

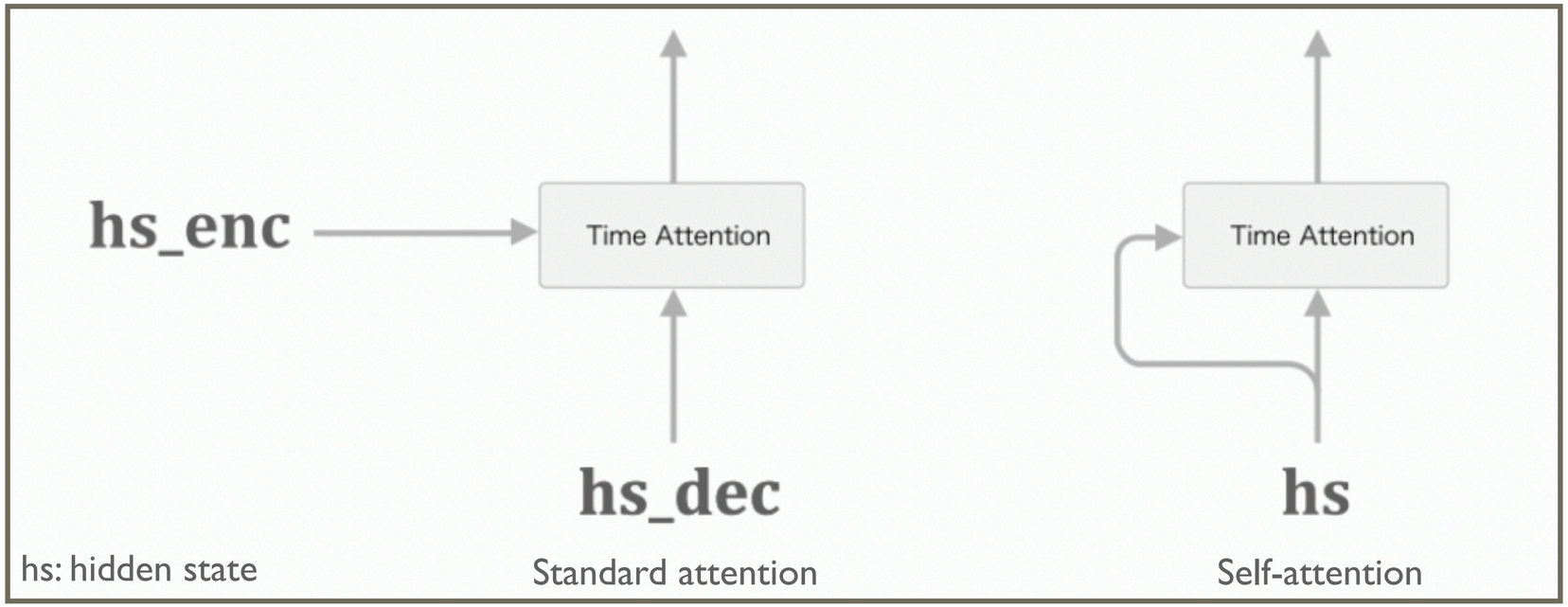
- 순차적인 특성이 유지되나, 정보간 거리에 따른 제약 (i.e., long-term dependency problem)을 지님
- 순차적인 특성 때문에 병렬처리를 할 수 없고, 계산속도가 느림



그림출처: https://colah.github.io/posts/2015-08-Understanding-LSTMs/

1. Introduction & Background

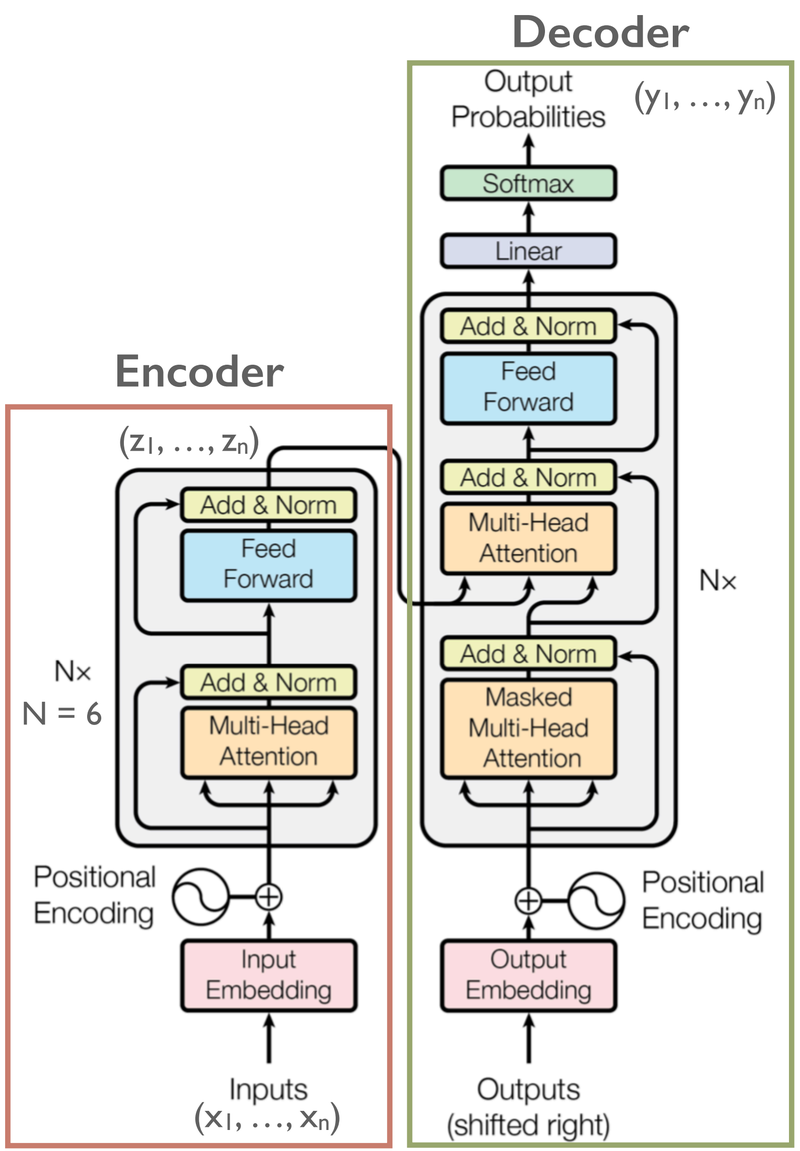
2. Attention

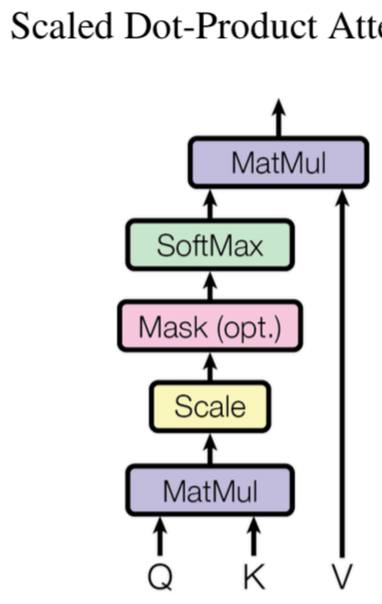


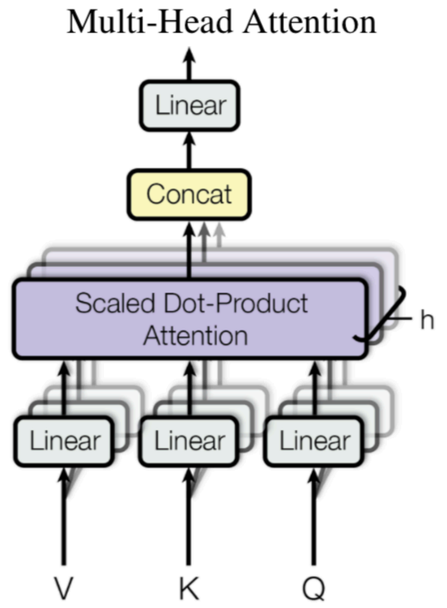
➡Transformer: Attention만을 사용한 모델로, 해서 상대적으로, 연산량이 줄고 성능도 매우 높음

그림 출처: 밑바닥부터 시작하는 딥러닝(2017), 개앞맵시 옮김, 한빛미디어.

2. Model Architecture







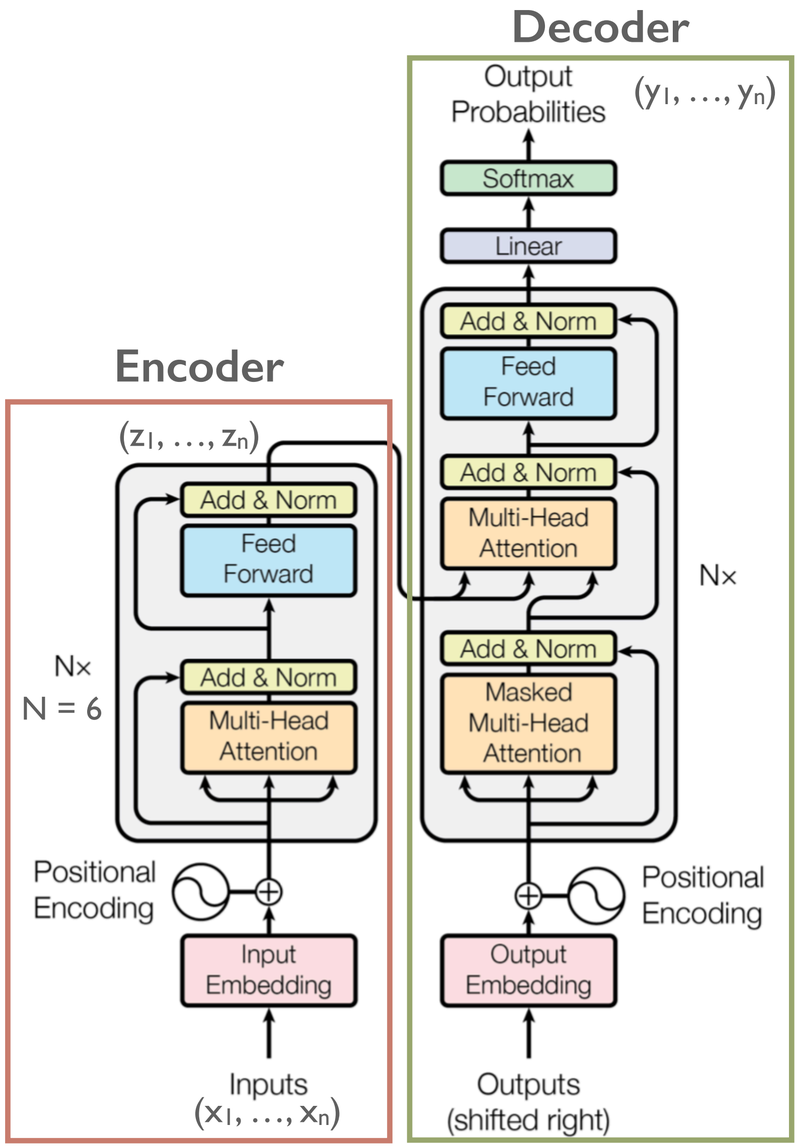
$$\text{Attention}(Q, K, V) = \text{softmax}(\frac{QK^T}{\sqrt{d_k}})V$$

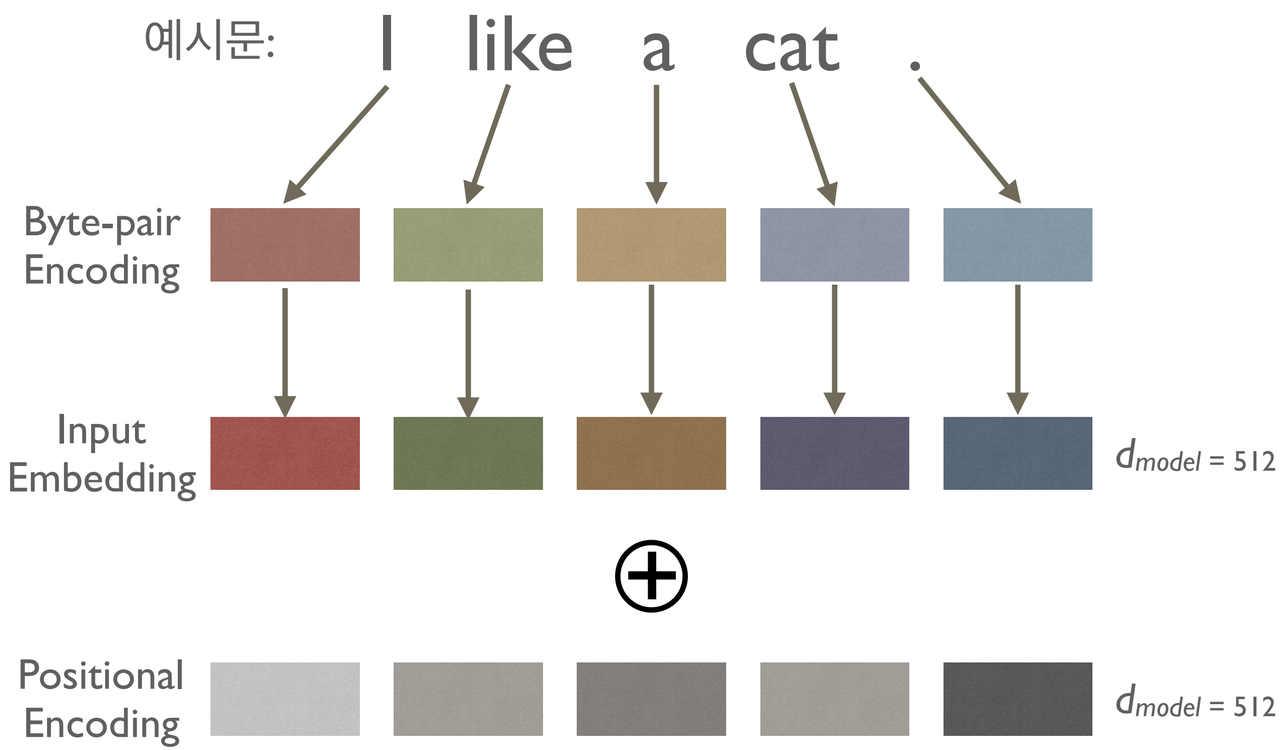
$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, ..., \text{head}_\text{h})W^O$$

$$\text{where head}_\text{i} = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

$$W_i^Q \in \mathbb{R}^{d_\text{model} \times d_k}, W_i^K \in \mathbb{R}^{d_\text{model} \times d_k}, W_i^V \in \mathbb{R}^{d_\text{model} \times d_v}, W^O \in \mathbb{R}^{hd_v \times d_\text{model}}$$

2. Model Architecture





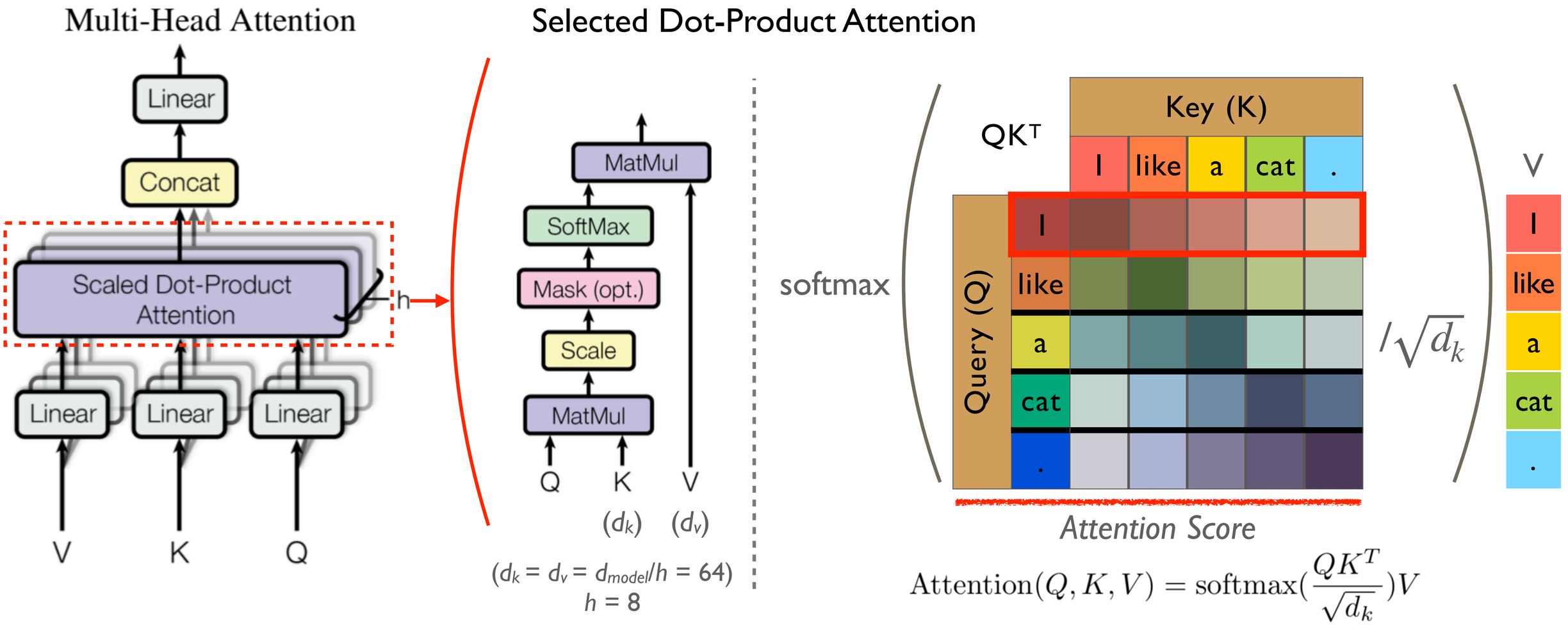
$$PE_{(pos,2i)} = sin(pos/10000^{2i/d_{\text{model}}})$$

$$PE_{(pos,2i+1)} = cos(pos/10000^{2i/d_{\text{model}}})$$

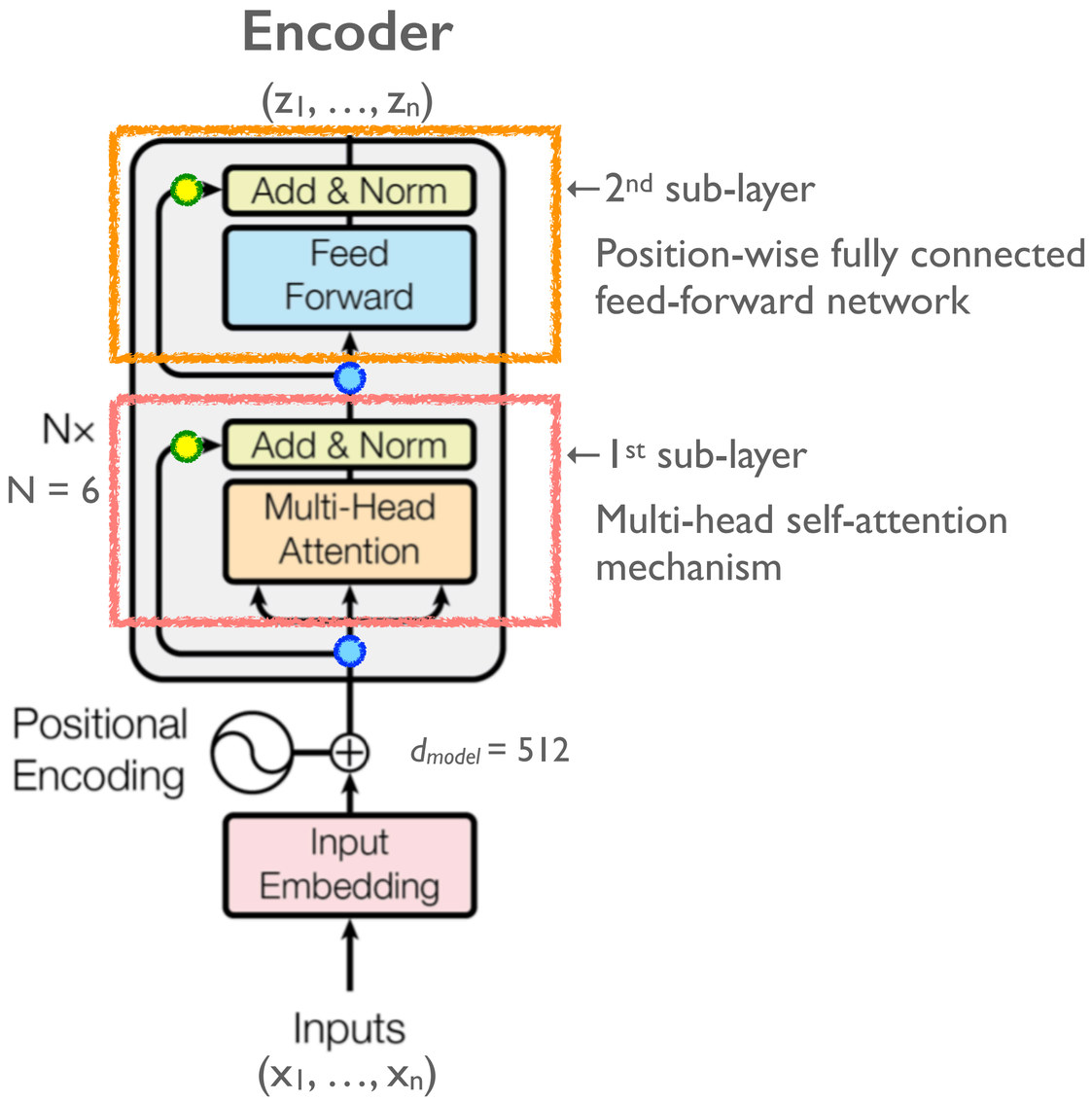
pos: position, *i*: dimension

2. Model Architecture

예시문:“I like a cat.”



2. Model Architecture



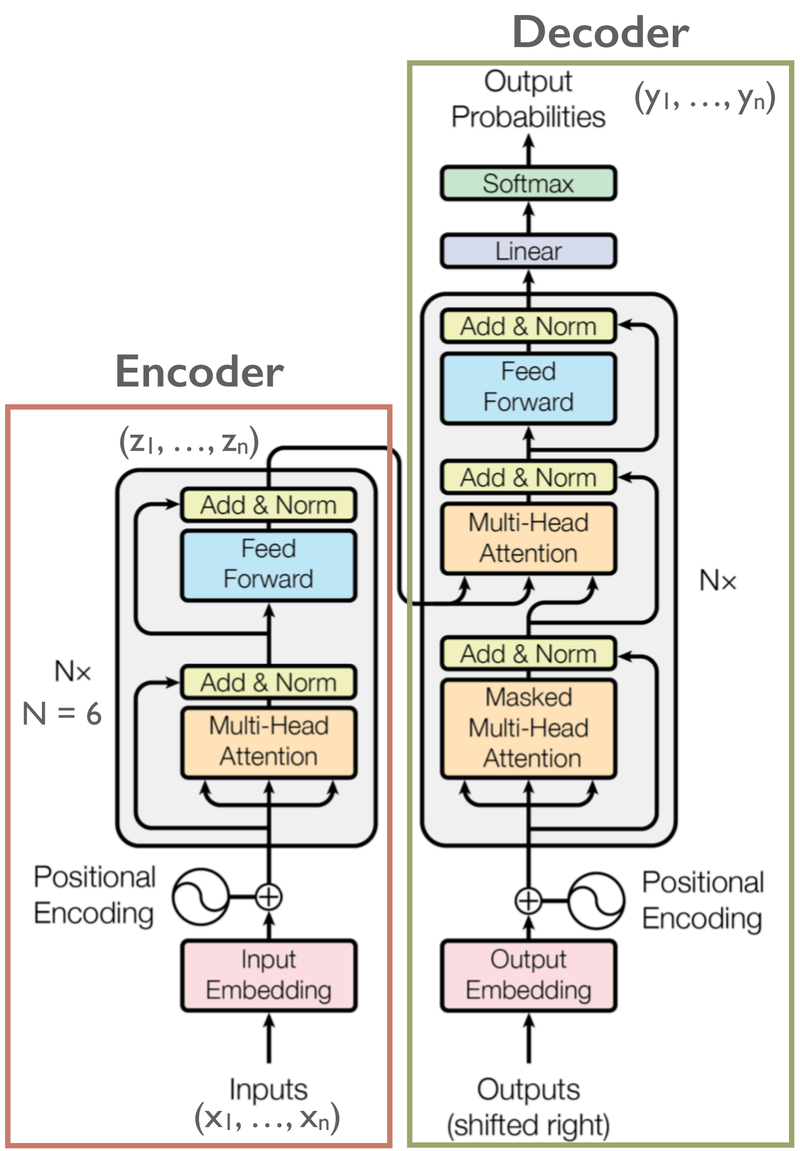
➡ Position-wise Feed-Forward Networks
- position separately and identically

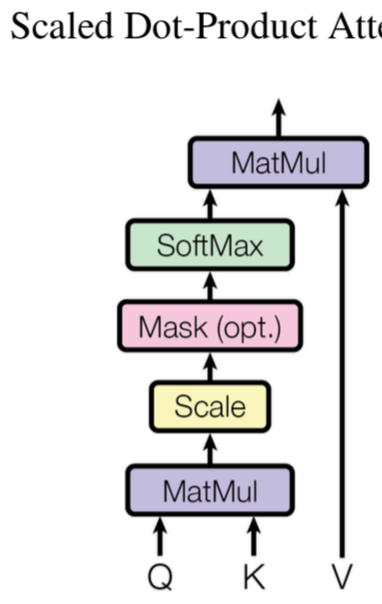
$$FFN(x) = \underbrace{\max\left(0, xW_1 + b_1\right)}_{\text{ReLU}} W_2 + b_2 \qquad (d_{ff} = 2{,}048)$$

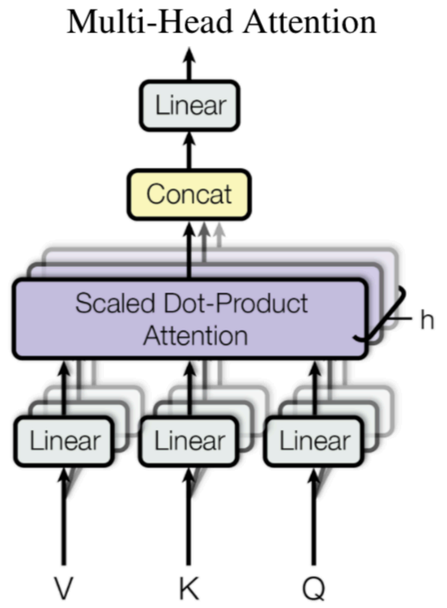
➡ Add & Norm
- Add: adding residual connection ●
- Norm: LayerNorm(x+Sublayer(x))
  (x = ●)

Norm: normalization

2. Model Architecture







$$\text{Attention}(Q, K, V) = \text{softmax}(\frac{QK^T}{\sqrt{d_k}})V$$

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, ..., \text{head}_\text{h})W^O$$

$$\text{where head}_\text{i} = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

$$W_i^Q \in \mathbb{R}^{d_\text{model} \times d_k}, W_i^K \in \mathbb{R}^{d_\text{model} \times d_k}, W_i^V \in \mathbb{R}^{d_\text{model} \times d_v}, W^O \in \mathbb{R}^{hd_v \times d_\text{model}}$$

Attention Visualizations

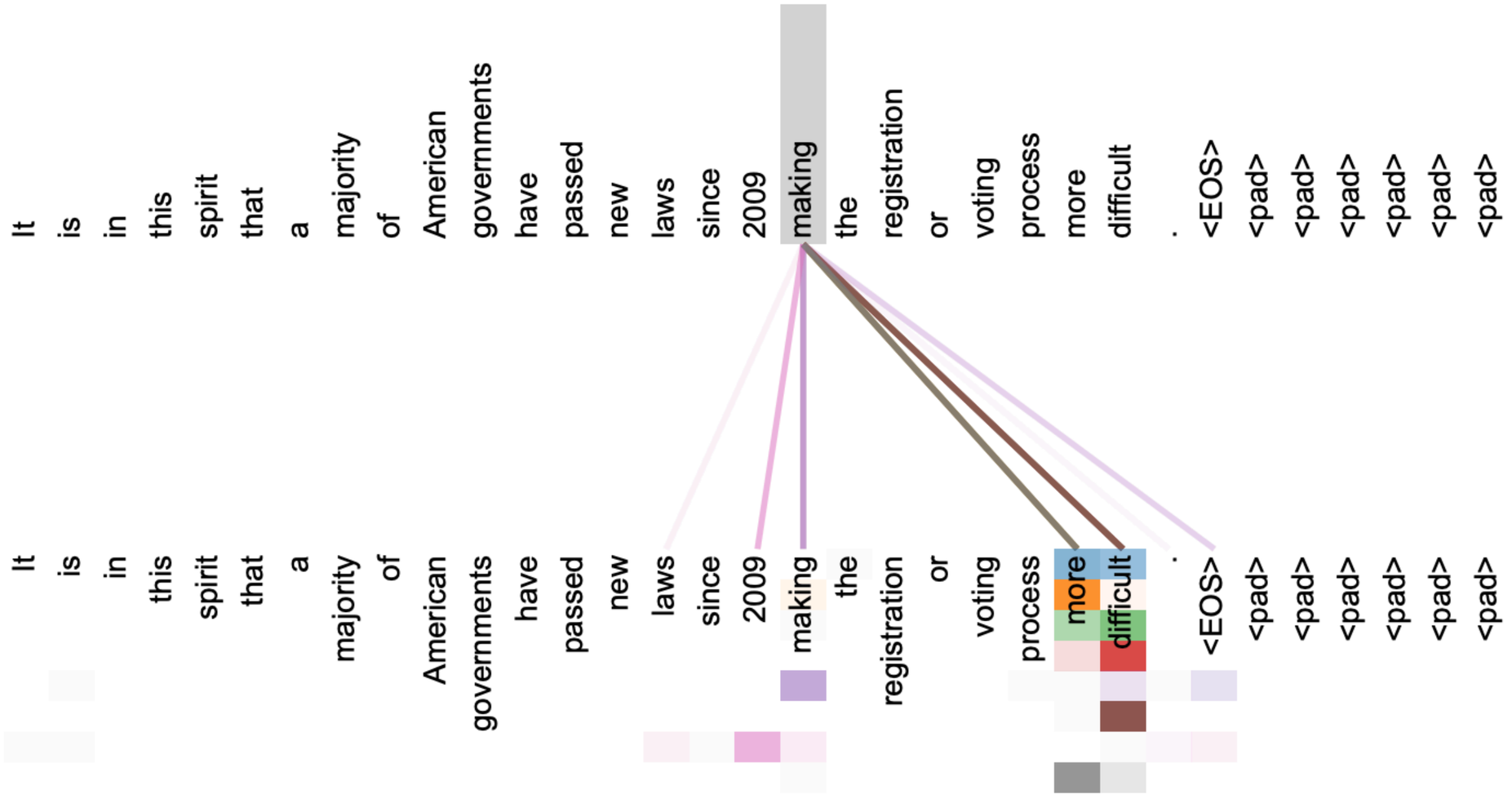


Figure 3: An example of the attention mechanism following long-distance dependencies in the encoder self-attention in layer 5 of 6. Many of the attention heads attend to a distant dependency of the verb 'making', completing the phrase 'making...more difficult'. Attentions here shown only for the word 'making'. Different colors represent different heads. Best viewed in color.

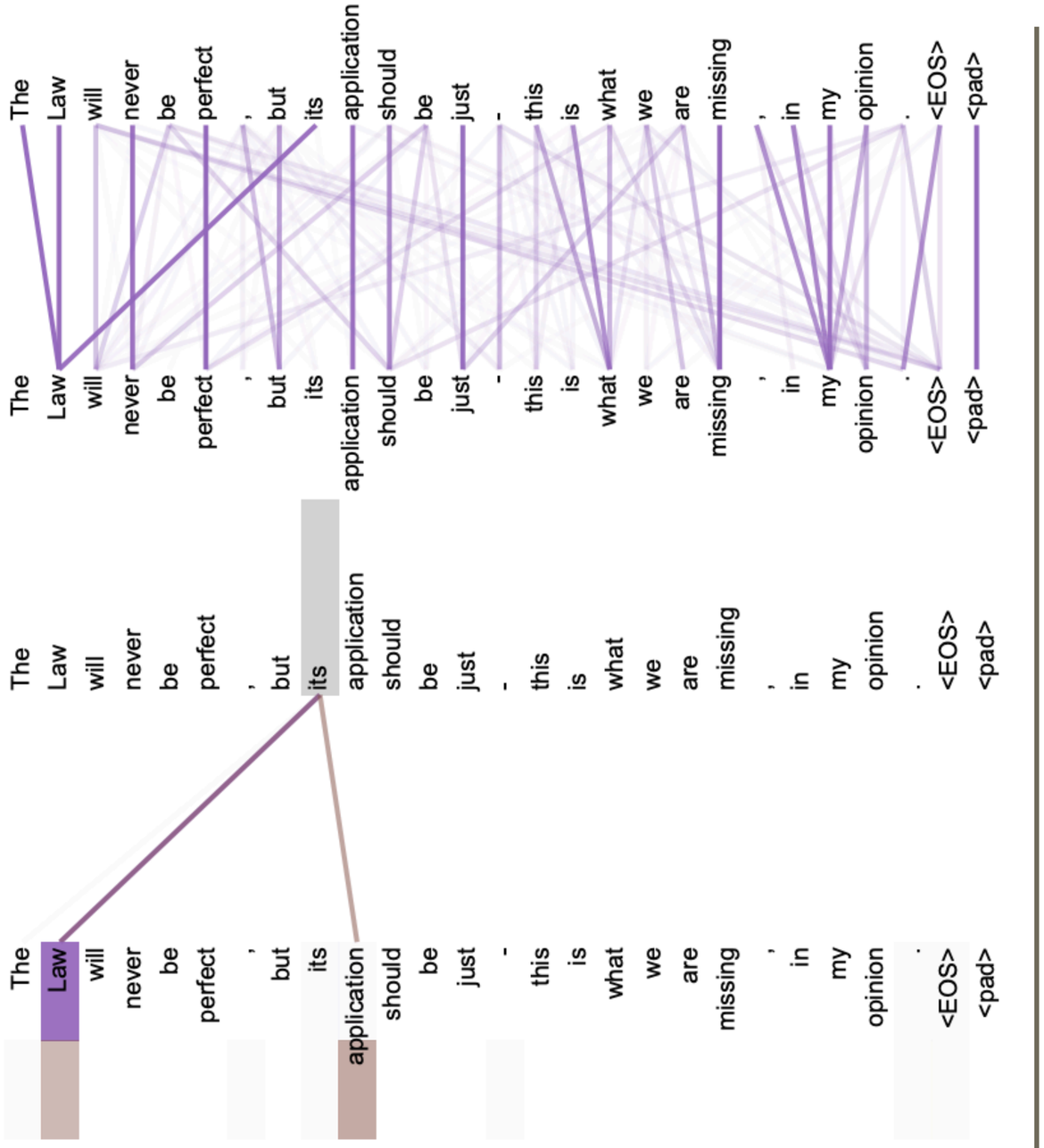


Figure 4: Two attention heads, also in layer 5 of 6, apparently involved in anaphora resolution. Top: Full attentions for head 5. Bottom: Isolated attentions from just the word 'its' for attention heads 5 and 6. Note that the attentions are very sharp for this word.

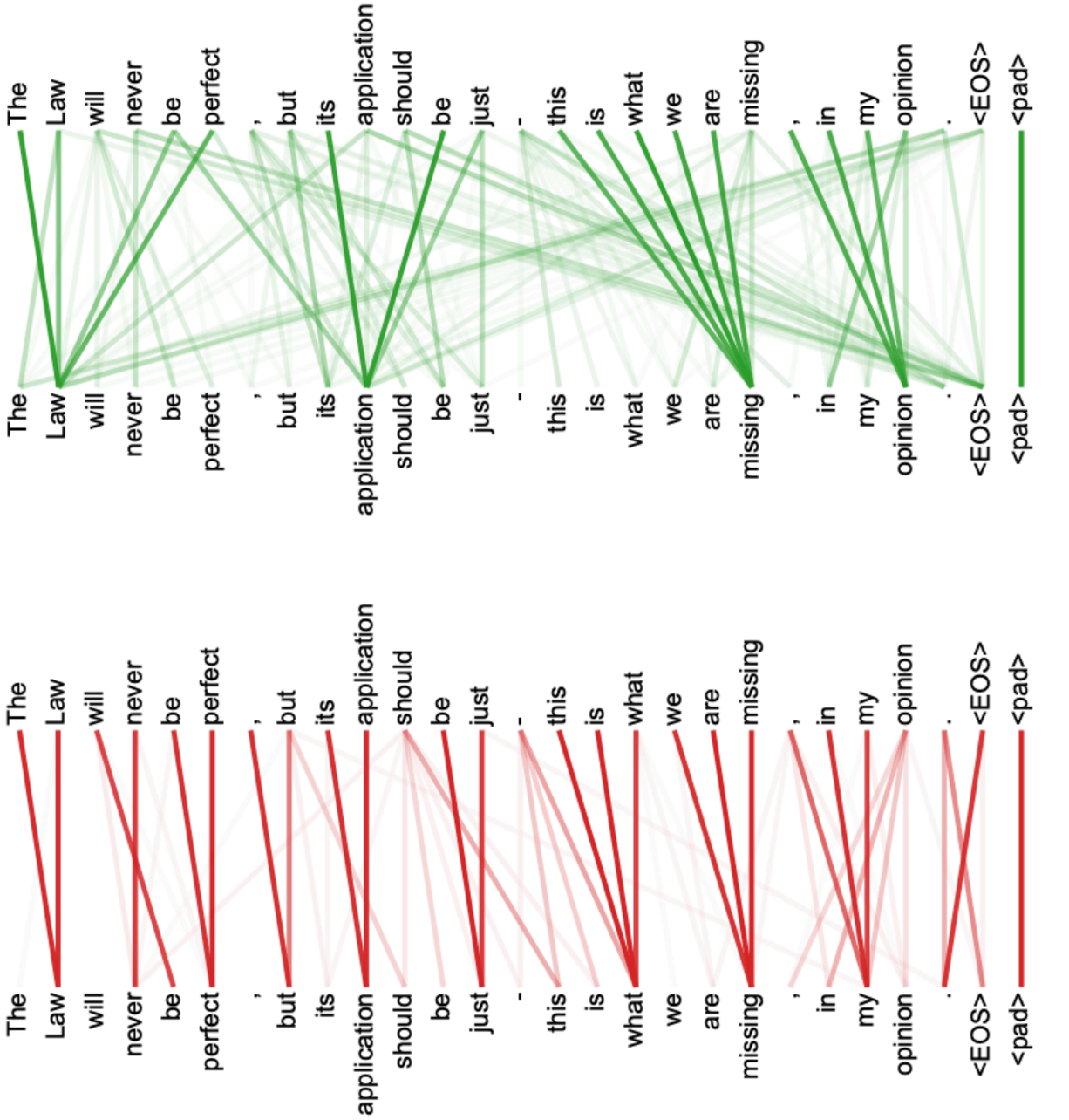
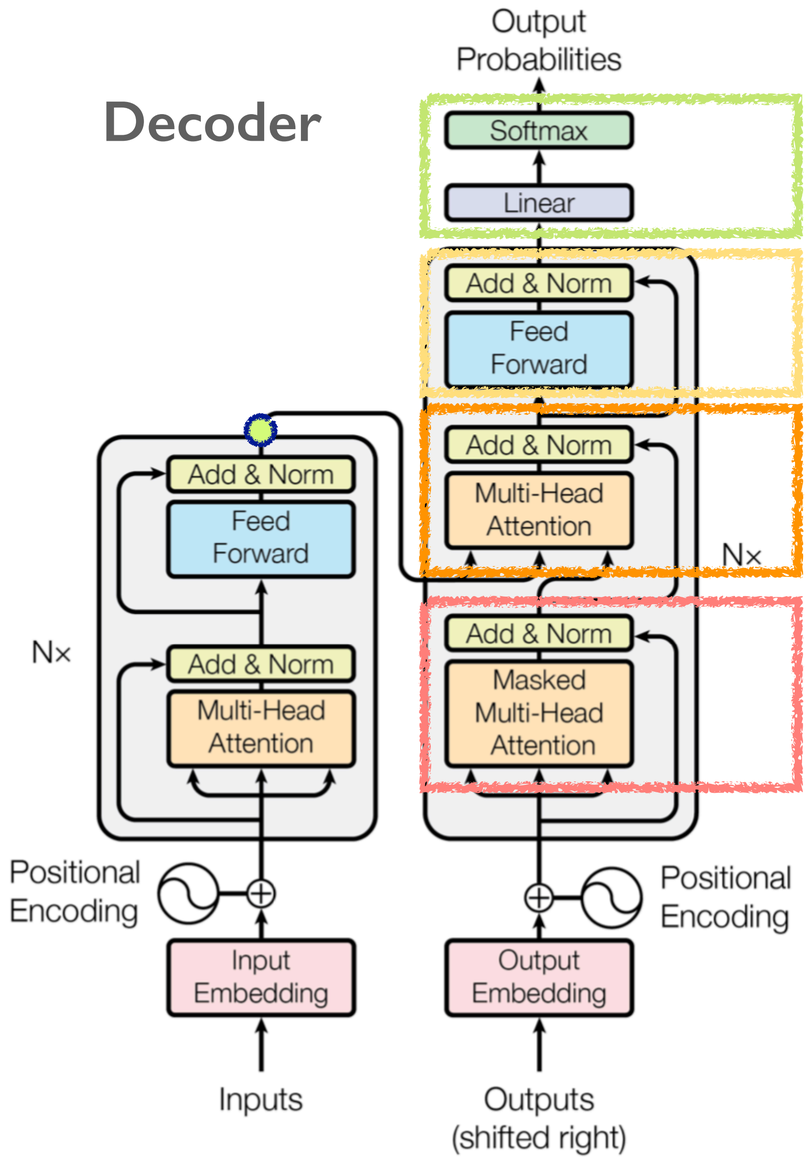


Figure 5: Many of the attention heads exhibit behaviour that seems related to the structure of the sentence. We give two such examples above, from two different heads from the encoder self-attention at layer 5 of 6. The heads clearly learned to perform different tasks.

2. Model Architecture

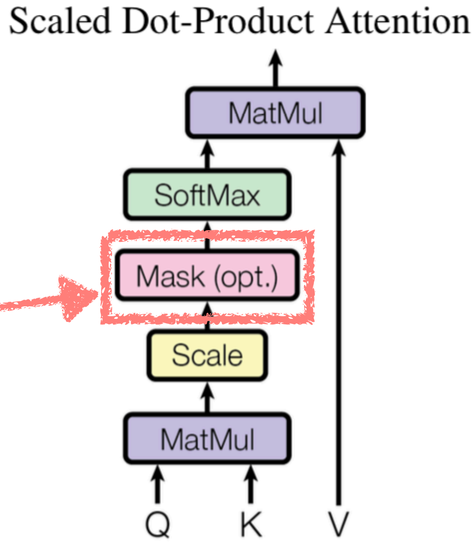


←3rd sub-layer
Position-wise fully connected feed-forward network

←2nd sub-layer
Multi-head self-attention mechanism

←1st sub-layer
Masked Multi-head self-attention mechanism

➡ 2nd sub-layer Input:
- Keys, Values
  : from output of encoder
- Queries
  : from previous decoder layer



➡Masking out (setting to -∞) all values in the input of the softmax which correspond to illegal connections.

3. Why Self-Attention

1. 레이어당 전체 연산량이 줄어든다.
2. 병렬화가 가능한 연산이 늘어난다.
3. Long-range의 term들의 dependency도 잘 학습할 수 있게 된다.
4. Attention을 사용하면 모델 자체의 동작을 해석하기 쉬워진다

Table 1: Maximum path lengths, per-layer complexity and minimum number of sequential operations for different layer types. n is the sequence length, d is the representation dimension, k is the kernel size of convolutions and r the size of the neighborhood in restricted self-attention.

Layer Type	Complexity per Layer	Sequential Operations	Maximum Path Length
Self-Attention	$O(n^2 \cdot d)$	$O(1)$	$O(1)$
Recurrent	$O(n \cdot d^2)$	$O(n)$	$O(n)$
Convolutional	$O(k \cdot n \cdot d^2)$	$O(1)$	$O(log_k(n))$
Self-Attention (restricted)	$O(r \cdot n \cdot d)$	$O(1)$	$O(n/r)$

4. Training

4.1. Training Data and Batching

- WMT 2014 English-French dataset [Medium]
  : 4.5 million sentence pairs (→ 37000 tokens)

```
iron cement is a ready for use paste which is laid as a fillet by putty knife or finger in the mould edges
iron cement protects the ingot against the hot , abrasive steel casting process .
a fire restant repair cement for fire places , ovens , open fireplaces etc .
Construction and repair of highways and ...
An announcement must be commercial character .
Goods and services advancement through the P.O.Box system is NOT ALLOWED .
Deliveries ( spam ) and other improper information deleted .
Translator Internet is a Toolbar for MS Internet Explorer .
It allows you to translate in real time any web pasge from one language to another .
You only have to select languages and TI does all the work for you ! Automatic dictionary updates ....
This software is written in order to increase your English keyboard typing speed , through teaching the bas
keyboard and give some training examples .
Each lesson teaches some extra keys , and there is also a practice , if it is chosen , one can practice the
previous lessons . The words chosen in the practice are mostly meaningful and relates to the tough keys ...
Are you one of millions out there who are trying to learn foreign language , but never have enough time ?
```

https://nlp.stanford.edu/projects/nmt/

- WMT 2014 English-G dataset
  : 36 million sentences (→ 32000 word-piece vocabulary)

- Training batch: 25000 source tokens and 25000 target tokens

4. Training

4.2. Hardware and Schedule

- Hardware: 8 NVIDIA P100 GPUs

- Schedule: [Each training step] 0.4 sec,
  [Base models; a total of 100,000 steps] 12 hrs, [Big models; 300,000 steps] 3.5 days

4.3. Optimizer

- Adam optimizer

$$lrate = d_{\text{model}}^{-0.5} \cdot \min(step\_num^{-0.5}, step\_num \cdot warmup\_steps^{-1.5}) \quad (3)$$

4.4. Regularization

- Residual Dropout: 1) applied to the output of each sub-layer, before it is added to the sub-layer input and normalized. 2) applied to the sums of the embeddings and the positional encodings in both the encoder and decoder. (P_{drop} = 0.1)

- Label Smoothing

5. Results

5.1. Machine Translation

Table 2: The Transformer achieves better BLEU scores than previous state-of-the-art models on the English-to-German and English-to-French newstest2014 tests at a fraction of the training cost.

Model	BLEU		Training Cost (FLOPs)	
	EN-DE	EN-FR	EN-DE	EN-FR
ByteNet [18]	23.75			
Deep-Att + PosUnk [39]		39.2		$1.0 \cdot 10^{20}$
GNMT + RL [38]	24.6	39.92	$2.3 \cdot 10^{19}$	$1.4 \cdot 10^{20}$
ConvS2S [9]	25.16	40.46	$9.6 \cdot 10^{18}$	$1.5 \cdot 10^{20}$
MoE [32]	26.03	40.56	$2.0 \cdot 10^{19}$	$1.2 \cdot 10^{20}$
Deep-Att + PosUnk Ensemble [39]		40.4		$8.0 \cdot 10^{20}$
GNMT + RL Ensemble [38]	26.30	41.16	$1.8 \cdot 10^{20}$	$1.1 \cdot 10^{21}$
ConvS2S Ensemble [9]	26.36	**41.29**	$7.7 \cdot 10^{19}$	$1.2 \cdot 10^{21}$
Transformer (base model)	27.3	38.1	**$3.3 \cdot 10^{18}$**	
Transformer (big)	**28.4**	**41.8**	$2.3 \cdot 10^{19}$	

5. Results

5.2. Model Variations

Table 3: Variations on the Transformer architecture. Unlisted values are identical to those of the base model. All metrics are on the English-to-German translation development set, newstest2013. Listed perplexities are per-wordpiece, according to our byte-pair encoding, and should not be compared to per-word perplexities.

	N	d_{model}	d_{ff}	h	d_k	d_v	P_{drop}	ϵ_{ls}	train steps	PPL (dev)	BLEU (dev)	params $\times 10^6$
base	6	512	2048	8	64	64	0.1	0.1	100K	4.92	25.8	65
(A)				1	512	512				5.29	24.9	
				4	128	128				5.00	25.5	
				16	32	32				4.91	25.8	
				32	16	16				5.01	25.4	
(B)					16					5.16	25.1	58
					32					5.01	25.4	60
(C)	2									6.11	23.7	36
	4									5.19	25.3	50
	8									4.88	25.5	80
		256			32	32				5.75	24.5	28
		1024			128	128				4.66	26.0	168
			1024							5.12	25.4	53
			4096							4.75	26.2	90
(D)							0.0			5.77	24.6	
							0.2			4.95	25.5	
								0.0		4.67	25.3	
								0.2		5.47	25.7	
(E)	positional embedding instead of sinusoids									4.92	25.7	
big	6	1024	4096	16			0.3		300K	**4.33**	**26.4**	213

5. Results

5.3. English Constituency Parsing

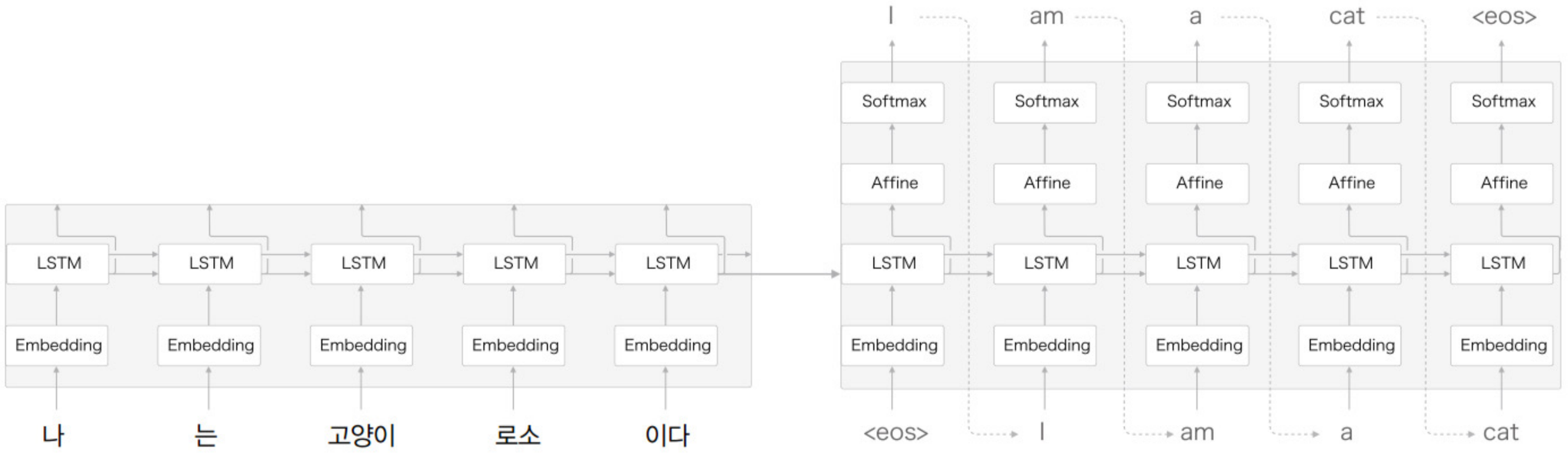
Table 4: The Transformer generalizes well to English constituency parsing (Results are on Section 23 of WSJ)

Parser	Training	WSJ 23 F1
Vinyals & Kaiser el al. (2014) [37]	WSJ only, discriminative	88.3
Petrov et al. (2006) [29]	WSJ only, discriminative	90.4
Zhu et al. (2013) [40]	WSJ only, discriminative	90.4
Dyer et al. (2016) [8]	WSJ only, discriminative	91.7
Transformer (4 layers)	WSJ only, discriminative	91.3
Zhu et al. (2013) [40]	semi-supervised	91.3
Huang & Harper (2009) [14]	semi-supervised	91.3
McClosky et al. (2006) [26]	semi-supervised	92.1
Vinyals & Kaiser el al. (2014) [37]	semi-supervised	92.1
Transformer (4 layers)	semi-supervised	92.7
Luong et al. (2015) [23]	multi-task	93.0
Dyer et al. (2016) [8]	generative	93.3

6. Conclusion

- Transformer: Recurrent, convolution을 사용하지 않고, attention만 사용한 모델
- 다른 모델들 보다 훨씬 빠른 학습속도 그리고 좋은 성능을 지님
- 번역 뿐만 아니라 이미지 등 큰 입력을 갖는 분야에도 적용될 것이 기대됨

Seq2Seq



(Seq2seq 전체 구조)

Seq2Seq는 Encorder으로 고정 길이 벡터로 변환한 것을 Decorder가 받아서 다시 원래의 정보로 되돌리는 구조
(Seq2seq는 Encoder-Decoder모델 이라고도 부른다)

BLEU Score

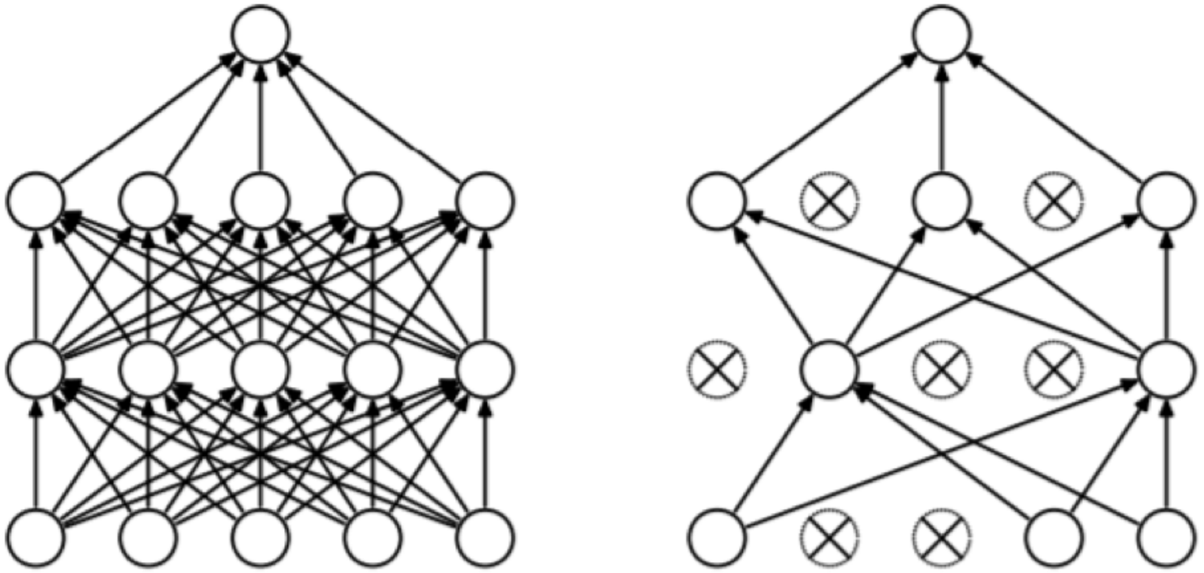
- BLEU (Bilingual Evaluation Understudy) score
  - 번역을 하는 모델에 주요 사용되는 성과지표
    - n-gram을 통한 순서쌍들이 얼마나 겹치는지 측정(precision)
    - 문장길이에 대한 과적합 보정 (Brevity Penalty)
    - 같은 단어가 연속적으로 나올 때 과적합되는 것을 보정 (Clipping)

$$BLEU = min(1, \frac{output\ length(\text{예측 문장})}{reference\ length(\text{실제 문장})})(\prod_{i=1}^{4} precision_i)^{\frac{1}{4}}$$

➡ 정답과 번역된 문장 사이의 공통 단어 수를 이용하여 품질 측정

참고 사이트: https://donghwa-kim.github.io/BLEU.html

Dropout



(a) Standard Neural Net (b) After applying dropout.

(그림 왼쪽은 일반 신경망, 오른쪽은 드롭아웃을 적용한 신경망)

- Overfitting 억제 방법으로 가중치 감소가 있지만, 신경망 모델이 복잡해지면 가중치 감소만으로는 대응이 어려워진다.
  이 때 이용하는 것이 Dropout 이다.
- Dropout: 뉴런을 임의로 삭제하며 학습하는 방법으로, 훈련 때 은닉층의 뉴런을 무작위로 골라 삭제한다.
  삭제한 뉴런은 신호를 전달하지 않게 된다. 훈련때는 데이터를 흘릴 때마다 삭제할 뉴런을 무작위로 선택하고, 시험때는 모든 뉴런에 신호를 전달. 단, 시험 때는 각 뉴런의 출력에 훈련 때 삭제한 비율을 곱하여 출력.